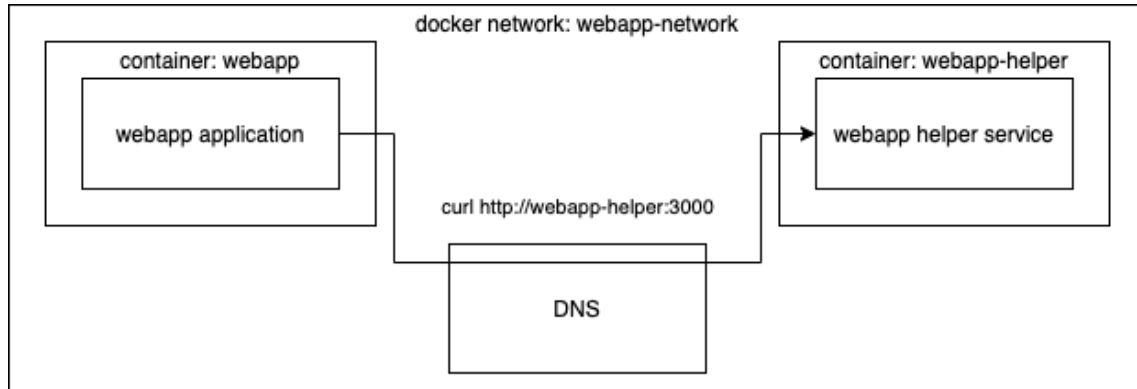


Подключение двух сервисов, таких как сервер и его база данных, в docker можно достичь с помощью [Docker network](#). В дополнение к запуску сервисов, перечисленных в docker-compose.yml, Docker Compose автоматически создает и объединяет оба контейнера в сеть с [DNS](#). Каждый сервис называется по имени, данному в файле docker-compose.yml. Таким образом, контейнеры могут ссылаться друг на друга просто по их именам сервисов, что отличается от имени контейнера.



Здесь два сервиса в одной сети: webapp и webapp-helper. Webapp-helper имеет сервер, прослушивающий запросы на порту 3000, к которому webapp хочет получить доступ. Поскольку они были определены в одном файле docker-compose.yml, доступ тривиален. Docker Compose уже позаботился о создании сети, и webapp может просто отправить запрос на webapp-helper:3000, внутренний DNS переведет это в правильный доступ, и порты не нужно публиковать за пределы сети.

...tip Напоминание о безопасности: Планируйте вашу инфраструктуру и придерживайтесь вашего плана

В следующем упражнении, и в некоторых последующих упражнениях, есть иллюстрация инфраструктуры. Посмотрите на нее и используйте ее для написания конфигурации.

Например, в Упражнении 2.4 мы не хотим открывать порты к Redis для внешнего мира. Не добавляйте конфигурацию `ports` под Redis! Backend сможет получить доступ к приложению внутри Docker сети.

...

Упражнение 2.4

...info Упражнение 2.4

В этом упражнении вы должны расширить конфигурацию, сделанную в [Упражнении 2.3](#), и настроить пример backend для использования базы данных ключ-значение [Redis](#).

Redis довольно часто используется как [кэш](#) для хранения данных, чтобы будущие запросы данных могли обслуживаться быстрее.

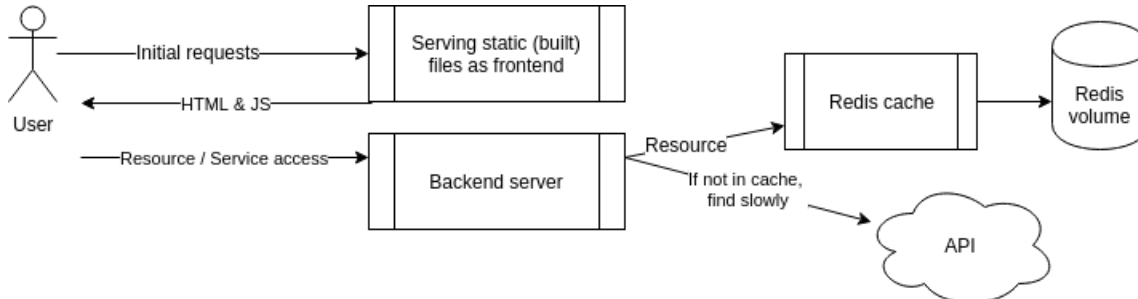
Backend использует медленный API для получения некоторой информации. Вы можете протестировать медленный API, запросив `/ping?redis=true` с помощью curl. Frontend приложение имеет кнопку для тестирования этого.

Итак, вы должны улучшить производительность приложения и настроить контейнер Redis для кэширования информации для backend. [Документация](#) образа Redis может содержать полезную информацию.

Backend [README](#) должен содержать всю информацию, необходимую для настройки backend.

Когда вы правильно настроите, кнопка станет зеленой.

Отправьте docker-compose.yml



Конфигурация [restart: unless-stopped](#) может помочь, если Redis требуется некоторое время для готовности.

:::

Ручное определение сети

Также возможно определить сеть вручную в файле Docker Compose. Основное преимущество ручного определения сети заключается в том, что это упрощает настройку конфигурации, где контейнеры, определенные в двух разных файлах Docker Compose, совместно используют сеть, и могут легко взаимодействовать друг с другом.

Давайте теперь посмотрим, как сеть определяется в docker-compose.yml:

```
version: "3.8"

services:
  db:
    image: postgres:13.2-alpine
    networks:
      - database-network # Name in this Docker Compose file

networks:
  database-network: # Name in this Docker Compose file
    name: database-network # Name that will be the actual name of the network
```

Это определяет сеть с именем `database-network`, которая создается с помощью `docker compose up` и удаляется с помощью `docker compose down`.

Как видно, сервисы настраиваются на использование сети, добавляя `networks` в определение сервиса.

Установление соединения с внешней сетью (то есть сетью, определенной в другом docker-compose.yml, или созданной другим способом) выполняется следующим образом:

```
version: "3.8"

services:
  db:
    image: backend-image
    networks:
      - database-network

networks:
  database-network:
    external:
      name: database-network # Must match the actual name of the network
```

По умолчанию все сервисы добавляются в сеть с именем `default`. Сеть по умолчанию можно настроить, и это позволяет подключаться к внешней сети по умолчанию:

```
version: "3.8"

services:
  db:
    image: backend-image

networks:
  default:
    external:
      name: database-network # Must match the actual name of the network
```

Масштабирование

Compose также может масштабировать сервис для запуска нескольких экземпляров:

```
$ docker compose up --scale whoami=3

WARNING: The "whoami" service specifies a port on the host. If multiple containers
for this service are created on a single host, the port will clash.

Starting whoami_whoami_1 ... done
Creating whoami_whoami_2 ... error
Creating whoami_whoami_3 ... error
```

Команда завершается ошибкой из-за конфликта портов, так как каждый экземпляр будет пытаться привязаться к одному и тому же порту хоста (8000).

Мы можем обойти это, указав только порт контейнера. Как упоминалось в [части 1](#), при неуказанном порте хоста Docker автоматически выберет свободный порт.

Обновите определение портов в `docker-compose.yml`:

```
ports:
```

```
- 8000
```

Затем запустите команду снова:

```
$ docker compose up --scale whoami=3
Starting whoami_whoami_1 ... done
Creating whoami_whoami_2 ... done
Creating whoami_whoami_3 ... done
```

Все три экземпляра теперь работают и прослушивают случайные порты хоста. Мы можем использовать `docker compose port`, чтобы узнать, к каким портам привязаны экземпляры.

```
$ docker compose port --index 1 whoami 8000
0.0.0.0:32770

$ docker compose port --index 2 whoami 8000
0.0.0.0:32769

$ docker compose port --index 3 whoami 8000
0.0.0.0:32768
```

Теперь мы можем делать curl запросы к этим портам:

```
$ curl 0.0.0.0:32769
I'm 536e11304357

$ curl 0.0.0.0:32768
I'm 1ae20cd990f7
```

В серверной среде у вас часто будет [балансировщик нагрузки](#) перед сервисом. Для контейнеризованной локальной среды (или одного сервера) одним из хороших решений является использование <https://github.com/jwilder/nginx-proxy>.

Давайте добавим nginx-proxy в наш compose файл и удалим привязки портов из сервиса whoami. Мы смонтируем наш [docker.sock](#) (сокет, который используется для связи с [Docker Daemon](#)) внутри контейнера в режиме `:ro` только для чтения:

```
version: "3.8"

services:
  whoami:
    image: jwilder/whoami
  proxy:
    image: jwilder/nginx-proxy
  volumes:
    - /var/run/docker.sock:/tmp/docker.sock:ro
  ports:
    - 80:80
```

Давайте протестируем конфигурацию:

```
$ docker compose up -d --scale whoami=3
$ curl localhost:80
<html>
<head><title>503 Service Temporarily Unavailable</title></head>
<body bgcolor="white">
<center><h1>503 Service Temporarily Unavailable</h1></center>
<hr><center>nginx/1.13.8</center>
</body>
</html>
```

Оно "работает", но Nginx просто не знает, какой сервис мы хотим. `nginx-proxy` работает с двумя переменными окружения: `VIRTUAL_HOST` и `VIRTUAL_PORT`. `VIRTUAL_PORT` не нужен, если сервис имеет `EXPOSE` в своем Docker образе. Мы можем видеть, что `jwtilder/whoami` устанавливает его: <https://github.com/jwtilder/whoami/blob/master/Dockerfile#L9>

- Примечание: Пользователи Mac с процессором M1 могут увидеть следующее сообщение об ошибке: `runtime: failed to create new OS thread`. В этом случае вы можете использовать Docker Image `ninanung/nginx-proxy` вместо него, который предлагает временное исправление, пока `jwtilder/nginx-proxy` не будет обновлен для поддержки Mac M1.

Домен `colasloth.com` настроен так, что все поддомены указывают на `127.0.0.1`. Больше информации о том, как это работает, можно найти на colasloth.github.io, но кратко это простой DNS "хак". Существует несколько других доменов, служащих той же цели, таких как `localtest.me`, `lvh.me` и `vsar.me`, чтобы назвать несколько. В любом случае, давайте используем `colasloth.com` здесь:

```
version: "3.8"

services:
  whoami:
    image: jwtilder/whoami
    environment:
      - VIRTUAL_HOST=whoami.colasloth.com
  proxy:
    image: jwtilder/nginx-proxy
    volumes:
      - /var/run/docker.sock:/tmp/docker.sock:ro
    ports:
      - 80:80
```

Теперь прокси работает:

```
$ docker compose up -d --scale whoami=3
$ curl whoami.colasloth.com
I'm f6f85f4848a8
$ curl whoami.colasloth.com
I'm 740dc0de1954
```

Давайте добавим еще пару контейнеров за тем же прокси. Мы можем использовать официальный образ `nginx` для обслуживания простой статической веб-страницы. Нам даже не нужно собирать образы контейнеров, мы можем просто смонтировать содержимое в образ. Давайте подготовим некоторое содержимое для двух сервисов с именами "hello" и "world".

```
$ echo "hello" > hello.html
$ echo "world" > world.html
```

Затем добавьте эти сервисы в файл `docker-compose.yml`, где вы монтируете только содержимое как `index.html` в путь `nginx` по умолчанию:

```
hello:
  image: nginx:1.19-alpine
  volumes:
    - ./hello.html:/usr/share/nginx/html/index.html:ro
  environment:
    - VIRTUAL_HOST=hello.colasloth.com
world:
  image: nginx:1.19-alpine
  volumes:
    - ./world.html:/usr/share/nginx/html/index.html:ro
  environment:
    - VIRTUAL_HOST=world.colasloth.com
```

Теперь давайте протестируем:

```
$ docker compose up -d --scale whoami=3
$ curl hello.colasloth.com
hello

$ curl world.colasloth.com
world

$ curl whoami.colasloth.com
I'm f6f85f4848a8

$ curl whoami.colasloth.com
I'm 740dc0de1954
```

Теперь у нас есть базовая настройка хостинга на одной машине, работающая.

Протестируйте обновление `hello.html` без перезапуска контейнера, это работает?

Упражнения 2.5

:::info Упражнение 2.5

Проект <https://github.com/docker-hy/material-applications/tree/main/scaling-exercise> — это едва работающее приложение. Идите вперед и клонируйте его для себя. Проект уже включает `docker-compose.yml`, так что вы можете запустить его, запустив `docker compose up`.

Приложение должно быть доступно через <http://localhost:3000>. Однако оно работает недостаточно хорошо, и мы добавили балансировщик нагрузки для масштабирования. Ваша задача — масштабировать контейнеры `compute`, чтобы кнопка в приложении стала зеленой.

Это упражнение было создано с помощью [Sasu Mäkinen](#)

Пожалуйста, верните использованные команды для этого упражнения.

:::